

Core Computer Science

Interview Cracker question bank

Generated from the Markdown sources in the Interview Cracker repository. Every code snippet in these pages was executed and checked before publication.

Concurrency

Where correct-looking code is wrong 1% of the time, on a machine you cannot reproduce.

[Easy] Q. What is a race condition?

Answer. When the result depends on the unpredictable timing of concurrent operations.

The canonical example is that `count += 1` is not one operation. It is three:

```
LOAD  count -> register
ADD    1
STORE register -> count
```

Two threads can both load 5, both compute 6, and both store 6. Two increments, one net effect - a **lost update**. Worked demonstration: `examples/rust/concurrency/threads_and_channels.rs`.

The general term is a **critical section**: a region that must not be entered by more than one thread at a time.

[Easy] Q. What is a mutex, and how does it differ from a semaphore?

Answer. A **mutex** enforces mutual exclusion: one holder at a time, and ownership matters - the thread that locks it must be the one that unlocks it.

A **semaphore** is a counter permitting **N** concurrent holders. `acquire` decrements (blocking at zero), `release` increments. A semaphore with **N=1** resembles a mutex but has no ownership, so any thread may release it - which makes it usable as a signal between threads, and easier to misuse.

Use a mutex to protect shared state. Use a semaphore to limit a resource pool ("at most 10 concurrent database connections").

[Medium] Q. What is a deadlock in application code, and how do you avoid it?

Answer. Two threads each holding a lock the other needs.

```
# Thread 1          # Thread 2
with lock_a:        with lock_b:
    with lock_b:    with lock_a:    # both block forever
    ...             ...
```

The reliable fix is **consistent lock ordering** - every thread acquires locks in one global order (say, by object id), which makes a cycle impossible. Also: prefer a single coarser lock over two fine ones, use timeouts (`tryLock`) so a deadlock becomes a recoverable error, and never hold a lock across a network call.

[Medium] Q. What does `volatile` / `atomic` mean, and is it enough?

Answer. They solve two different problems, and conflating them is the common error.

Visibility. Without synchronisation, a value written by one thread may sit in a CPU cache or register and never become visible to another. Java's `volatile` forces reads and writes to go to main memory. It does **not** make compound operations safe - `volatile int count; count++` is still a race.

Atomicity. An atomic operation completes indivisibly. `AtomicInteger.incrementAndGet()` performs the whole read-modify-write as one uninterruptible step, usually via a CPU compare-and-swap instruction.

So: `volatile` for a flag one thread writes and others read. **Atomics** for counters. A **mutex** when you must keep several variables consistent with each other - atomics protect one variable each, not an invariant across

two.

[Medium] Q. Async/await vs threads - when do you use which?

Answer. They solve different bottlenecks.

Threads give you parallelism across cores. Right for **CPU-bound** work. Costly: ~1 MB of stack each, plus context switches.

Async gives you concurrency on (usually) one thread by suspending a task at its I/O points so another can run. Right for **I/O-bound** work - thousands of in-flight requests, each mostly waiting. Cheap: a suspended task is a small heap object, not a stack.

The rule: **async for waiting, threads for computing**. And the trap that follows from it - a blocking or CPU-heavy call inside an async function stalls the whole event loop and every other task on it. Offload that work to a thread pool.

[Medium] Q. What is the difference between a thread pool and creating threads on demand?

Answer. A pool creates a fixed set of worker threads once and feeds them tasks from a queue.

It fixes two problems with thread-per-task: **creation cost** (spawning a thread is expensive, and doing it per request wastes most of the work) and **unbounded growth** (10,000 simultaneous requests creating 10,000 threads will exhaust memory and thrash the scheduler).

The pool size is the tuning knob: roughly the core count for CPU-bound work (more threads just add switching); much higher for I/O-bound work, since most threads are blocked. A bounded task queue matters too - an unbounded one converts an overload into an out-of-memory error instead of applying backpressure. Worked example: `examples/go/concurrency/worker_pool.go`.

[Hard] Q. What is the producer-consumer problem?

Answer. Producers add items to a shared bounded buffer, consumers remove them. The coordination requirements:

- Producers must **block when the buffer is full** (backpressure).
- Consumers must **block when it is empty**.
- Mutations of the buffer must be mutually exclusive.

Implemented classically with a mutex plus two condition variables, or in modern code with a **blocking queue** (`BlockingQueue`, `queue.Queue`, a Go channel), which packages exactly this.

The point of the question is usually backpressure: if producers never block, a fast producer and a slow consumer grow the buffer until memory runs out. A bounded queue converts that failure into a slowdown, which is almost always what you want.

[Medium] Q. What is thread-safety, and how do you achieve it without locks?

Answer. Code is thread-safe if it behaves correctly when called concurrently, with no external synchronisation required.

Locks are one route. The others are usually better:

- **Immutability.** An object that never changes cannot be raced on. This is the
- **Confinement.** Do not share the state at all - give each thread its own copy
- **Atomics** for single-variable counters and flags.
- **Concurrent collections** - `ConcurrentHashMap` and friends, which lock

Rust enforces this at compile time: its borrow checker rejects a data race rather than letting you find it in production. See `examples/rust/ownership/count_word_frequencies_borrowed.rs`.

[Hard] Q. You use a compare-and-swap (CAS) loop to update a shared pointer

without a lock. A colleague says it's still racy. When would they be right?

Answer. They are right because of the **ABA problem**: CAS only compares the value, not the history. If another thread changes the pointer from A to B and back to A, your CAS sees "still A" and succeeds - even though the object it points at has been replaced (and possibly freed) in between. The value is the same, but the state your update assumed no longer is.

The canonical setting is a **lock-free stack**:

- 1 You read the head, it points to node **A** (A -> B -> C).
- 2 Another thread pops A and B, then a new allocation happens to reuse the
- 3 Your CAS compares "head is still A?" - yes - and links the new node after

The value (the address A) is identical; only the identity of what it points to changed. CAS cannot tell the difference.

The fixes, in order of how often they're the real answer:

- **Version / sequence stamp (the standard fix).** Store the pointer and a
- **Hazard pointers / epoch-based reclamation.** Prevent the real danger
- **Just take a lock.** Be honest about it. ABA is a cost you pay to be

The follow-up this sets up: "lock-free" and "wait-free" are different. A CAS loop can livelock (always losing the race to another thread) even if it never deadlocks - progress guarantees are a spectrum, and ABA is the clearest example of why the memory model, not just the algorithm, is what you actually have to get right.

Security & Cryptography

Interviewers use crypto questions to see if you can separate the three jobs that get lumped together: **confidentiality** (encryption), **integrity and authenticity** (hashes, MACs, signatures), and **availability** (DoS, rate limits). Getting the labels straight is most of the answer.

[Easy] Q. Symmetric vs asymmetric encryption - what's the difference and

why do we need both?

Answer. Symmetric (AES, ChaCha20) uses one shared key for encrypt and decrypt - fast, but you must get the key to the other party safely. **Asymmetric** (RSA, X25519) uses a public/private key pair - anyone can encrypt with the public key, only the holder of the private key can decrypt - which solves key distribution but is far slower.

Real systems use both: TLS does an **asymmetric** (or Diffie-Hellman) key exchange to agree on a fresh **symmetric** session key, then bulk-encrypts the data with the symmetric key. You get the secure key exchange of public keys and the speed of symmetric crypto. The same split underpins signing (sign fast with a hash, then apply the slow private key to the digest).

[Medium] Q. A cryptographic hash is not a MAC. What's the difference, and

why does it matter?

Answer. A plain hash (SHA-256) is a **one-way fingerprint**: it has no secret, so anyone who can see the message can recompute the digest. A **MAC** (HMAC) additionally takes a **shared secret key**, so only someone with the key can produce a valid tag - that's what gives it authenticity.

```
import hashlib, hmac, secrets

# Plain hash: deterministic, and reproducible by ANYONE who has the message.
assert hashlib.sha256(b"password").hexdigest() == \
    hashlib.sha256(b"password").hexdigest()

# HMAC: needs the key to forge a valid tag.
key = secrets.token_bytes(32)
msg = b"transfer 100 to bob"
tag = hmac.new(key, msg, hashlib.sha256).hexdigest()
assert tag == hmac.new(key, msg, hashlib.sha256).hexdigest() # same key, same tag
assert tag != hmac.new(key, b"transfer 200 to bob", hashlib.sha256).hexdigest()
assert hmac.compare_digest(tag, tag) and not hmac.compare_digest(tag, "x")
```

The concrete failure of `hash(secret + msg)` (a "homebrew MAC"): the concatenation is ambiguous, so different `(secret, msg)` pairs collide.

```
# Two different (secret, msg) splits hash to the SAME value:
a = hashlib.sha256(b"sec" + b"retmsg").hexdigest()
b = hashlib.sha256(b"secret" + b"msg").hexdigest()
assert a == b # both are the byte string 'secretmsg' - the split is unknowable
```

HMAC's structure (keyed inner/outer hash) is designed to avoid exactly this. Also note `hmac.compare_digest` - a **timing-safe** comparison, because a naive `==` can leak how many leading bytes of a forger's tag were correct.

[Medium] Q. How should you store user passwords?

Answer. Never a plain hash. Use a **slow, salted, keyed** function: a **per-user random salt** (so identical passwords produce different digests and rainbow tables are useless) plus a deliberately expensive function (bcrypt, scrypt, or Argon2 - or PBKDF2 with a high iteration count) so brute force is expensive.

```
import hashlib, secrets

password = b"hunter2"
salt = secrets.token_bytes(16) # per-user, stored alongside
digest = hashlib.pbkdf2_hmac("sha256", password, salt, 600_000).hex()

# Verification re-derives with the SAME salt and compares in constant time.
import hmac
assert hmac.compare_digest(
    hashlib.pbkdf2_hmac("sha256", password, salt, 600_000).hex(), digest
)
assert not hmac.compare_digest(
    hashlib.pbkdf2_hmac("sha256", b"hunter3", salt, 600_000).hex(), digest
)
# Same password, different salt -> different digest (defeats rainbow tables).
salt2 = secrets.token_bytes(16)
assert hashlib.pbkdf2_hmac("sha256", password, salt2, 600_000).hex() != digest
```

The three properties to name: **salt** (uniqueness), **slowness/work factor** (costs the attacker), and **you cannot reverse it** (so on breach an attacker still has to brute-force each password individually). Store only salt + digest, never the password or a reversible-encrypted form.

[Medium] Q. What happens during a TLS handshake, in one paragraph?

Answer. The client sends a `ClientHello` with its cipher preferences and a random; the server replies with its certificate (signed up a chain to a trusted CA) and its own random, and the two parties run a key exchange (ECDHE) from which both independently derive the same symmetric session keys using the two randoms. Because the server's certificate is verified against the CA chain, the client is confident it's talking to the right host, and because ECDHE contributes fresh randomness, the session keys are new each connection (giving **forward secrecy** - a later leak of the server's private key doesn't decrypt past sessions). Only then does encrypted application data flow, using the fast symmetric keys.

The two things interviewers probe: **authentication** (the certificate chain answers "are you really `bank.com`?") and **forward secrecy** (ECDHE answers "if my private key leaks next year, is today's traffic safe?").

[Hard] Q. An attacker steals your database. What should have been true,

and what are they still missing?

Answer. Layered defence, in order of how much it helps after the breach: (1) **passwords** were stored as salted slow hashes, so the dump doesn't hand over logins - the attacker must brute-force each one; (2) **sensitive fields** (cards, SSNs, tokens) were **encrypted at rest** with keys held outside the database (KMS/HSM), so the dump is ciphertext without the key; (3) **secrets** (API keys, DB credentials) were never in the repo or the image - they live in a secrets manager; (4) access was **least-privilege** and logged, limiting what the stolen role could reach.

The answer they want is the shift in mindset: you **cannot** assume the database is safe, so you make a stolen database worth less than the cost of attacking it. The single most-cited failure is storing passwords as a plain or fast hash (or reversible-encrypted), which turns a DB breach into an instant credential breach.

[Easy] Q. What's the difference between a hash, a MAC, and a signature?

Answer.

Tool	Secret?	Proves	Who can verify
Hash (SHA-256)	none	integrity only (accidental)	anyone
MAC (HMAC)	shared key	integrity + authenticity	anyone with the key
Signature (RSA/Ed25519)	private key	integrity + authenticity + non-repudiation	anyone with the public key

A **signature** is the asymmetric upgrade of a MAC: the sender signs with a private key, and anyone can verify with the public key, and the sender can't plausibly deny it (non-repudiation). A **MAC** is symmetric - fast, but the verifier must hold the shared secret, so it can't be publicly verified. Use MACs for server<->server over a trusted channel, signatures for publicly verifiable artifacts (code, containers, documents, TLS certs).

Networking

"What happens when you type a URL into a browser?" is the single most-asked open-ended interview question. Everything here feeds into it.

[Easy] Q. TCP vs UDP?

Answer.

	TCP	UDP
Connection	Handshake first	None - just send
Delivery	Guaranteed, retransmits	Best effort, may drop
Ordering	Preserved	None
Congestion control	Yes	No
Header	20+ bytes	8 bytes
Speed	Slower	Faster, lower latency

TCP when correctness matters: HTTP, email, file transfer, databases. **UDP** when timeliness beats completeness: live video and voice, gaming, DNS.

The insight worth stating: for a video call, a packet that arrives late is useless - retransmitting it would make things worse by delaying everything behind it. Dropping it and carrying on is the correct behaviour. (QUIC, which HTTP/3 runs on, is built on UDP and reimplements reliability per-stream to avoid exactly this head-of-line blocking.)

[Medium] Q. Explain the TCP three-way handshake, and why it is three.

Answer.

- 1 **SYN** - client sends its initial sequence number.
- 2 **SYN-ACK** - server acknowledges it and sends its own.
- 3 **ACK** - client acknowledges the server's.

Three steps because **both directions** must be established, and each side must confirm the other's sequence number. Two would leave the server unsure the client received its number.

The cost: one full round trip before any data flows. On a 100 ms link that is 100 ms before the request is even sent, plus another ~2 round trips for the TLS handshake. This is why connection reuse (keep-alive), HTTP/2 multiplexing and QUIC's 0-RTT resumption all exist.

Closing takes **four** steps (FIN/ACK each way), because each direction closes independently - one side can finish sending while still receiving.

[Easy] Q. What happens when you type a URL and press enter?

Answer. The structure matters more than the detail; go layer by layer.

- 1 **URL parsing** - scheme, host, port, path.
- 2 **DNS resolution** - browser cache -> OS cache -> router -> resolver -> root -> TLD
- 3 **TCP connection** - three-way handshake to that IP on port 443.

- 4 **TLS handshake** - negotiate the cipher, validate the certificate chain
- 5 **HTTP request** - GET / HTTP/1.1 plus headers (Host, cookies, Accept).
- 6 **Server processing** - load balancer -> application server -> cache or database
- 7 **Response** - status line, headers, body.
- 8 **Rendering** - parse HTML into the DOM, fetch CSS/JS/images (more requests,

Strong candidates then mention **caching at every layer** - DNS TTL, browser cache, CDN, server-side cache - because the fastest request is the one never made.

[Medium] Q. What is the difference between HTTP/1.1, HTTP/2 and HTTP/3?

Answer.

- **HTTP/1.1** - one request at a time per connection. Browsers open ~6
 - **HTTP/2** - binary framing with **multiplexing**: many concurrent streams over
 - **HTTP/3** - runs over **QUIC on UDP**, giving each stream independent delivery.
-

[Medium] Q. What does DNS actually do, and what is a TTL?

Answer. It translates a hostname into an IP address, through a hierarchy: root servers -> TLD servers (.com) -> the domain's authoritative nameservers.

Record types worth knowing: **A** (IPv4), **AAAA** (IPv6), **CNAME** (alias), **MX** (mail), **TXT** (verification, SPF), **NS** (delegation).

TTL is how long a resolver may cache a record. It is the trade-off at the heart of DNS-based failover: a low TTL means changes propagate fast but you pay more lookups; a high TTL is efficient but leaves stale records pointing at a dead server for hours. Lowering the TTL before a planned migration is the standard practice.

[Medium] Q. What does HTTPS actually protect, and how?

Answer. TLS provides three things:

- 1 **Encryption** - an eavesdropper sees ciphertext.
- 2 **Integrity** - tampering is detected.
- 3 **Authentication** - the certificate proves you are talking to the real host.

It works by using **asymmetric** cryptography (slow) only to agree on a shared **symmetric** key (fast), then encrypting the actual traffic symmetrically. Trust comes from a chain: the site's certificate is signed by an intermediate CA, signed by a root CA that your OS or browser already trusts.

Worth knowing what it does not hide: the destination IP, and the hostname during the handshake (SNI is sent in clear unless Encrypted Client Hello is in use). An observer cannot read your traffic but can see which site you visited.

[Easy] Q. What do the HTTP status code classes mean?

Answer.

Class	Meaning	Worth knowing
1xx	Informational	101 Switching Protocols (WebSocket upgrade)
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 permanent, 302 temporary, 304 Not Modified
4xx	Client error	400, 401 unauthenticated, 403 unauthorised, 404, 429 rate limited
5xx	Server error	500, 502 bad gateway, 503 unavailable, 504 timeout

The distinction interviewers probe: **401 vs 403** - 401 means "I do not know who you are, authenticate"; 403 means "I know who you are and you may not do this". And **301 vs 302** - 301 is cached aggressively by browsers and is effectively permanent, so it is hard to undo.

[Medium] Q. What is the difference between a load balancer at L4 and L7?

Answer. Layer 4 balances on TCP/UDP information - IP and port. It cannot see inside the request, so it is fast and protocol-agnostic, but it can only do simple distribution.

Layer 7 parses the HTTP request, so it can route on path, host, headers or cookies (`/api/*` to one pool, `/static/*` to another), terminate TLS, retry failed requests and do sticky sessions. The cost is CPU and latency.

Use L4 for raw throughput and non-HTTP protocols; use L7 when routing decisions depend on request content, which for a typical web application is most of the time.

[Medium] Q. TCP delivers reliably over a network that drops packets. How does it

detect that a segment is lost, and how does it avoid making the loss worse?

Answer. It detects loss two ways - a **retransmission timer** expires, or it receives **three duplicate ACKs** - and it avoids making things worse by **congestion control**: every loss is treated as a signal that the network is saturated, so the sender shrinks its rate instead of just retransmitting and holding steady.

Detection, the two mechanisms:

- **Retransmission timeout (RTO).** The sender measures RTT and sets a timer
- **Fast retransmit.** If the same ACK arrives three times, the missing

Congestion control, the part most answers skip:

- The sender maintains a **congestion window** (how many unacknowledged bytes)
- On any loss signal, the window is **halved** (Reno; CUBIC, the modern Linux)
- So the answer to "why not just retransmit at full speed?" is: the loss may

The follow-up this sets up: congestion control cannot distinguish a packet lost to **congestion** from one lost to a **flaky wireless link**. On a noisy Wi-Fi connection, the flow punishes itself for the radio's mistakes, which is one reason real-time and mobile protocols either tune these parameters hard or run over UDP with their own recovery (this is a large part of why QUIC puts loss recovery in user space where it can be changed without an

OS upgrade).

Operating Systems

Asked in almost every interview that touches backend or systems work.

[Easy] Q. Process vs thread?

Answer. A **process** is a running program with its own address space. A **thread** is an execution path inside a process; threads of one process share that address space.

	Process	Thread
Memory	Isolated	Shared (heap, globals, files)
Own stack	Yes	Yes
Creation cost	High	Low
Communication	IPC (pipes, sockets, shared memory)	Shared variables
Crash blast radius	Contained	Takes down the process

The shared address space is the entire trade-off: threads communicate cheaply, and that same sharing is what makes race conditions possible. Chrome uses a process per tab precisely so one crashed tab cannot take the browser with it.

[Medium] Q. What is a context switch and why is it expensive?

Answer. Saving the CPU state of one thread (registers, program counter, stack pointer) and restoring another's, so the CPU can run something else.

The direct cost - copying registers - is small, a microsecond or so. The real cost is indirect: the new thread's data is not in the CPU caches, so it runs slowly until they refill, and a process switch also flushes the TLB because the page tables change.

This is why thread-per-request servers stop scaling around a few thousand connections, and why async I/O exists - one thread multiplexing thousands of sockets never context-switches between them.

[Medium] Q. What is virtual memory?

Answer. An abstraction giving each process its own contiguous address space, which the MMU translates to physical addresses via **page tables**.

It provides three things at once:

- **Isolation** - a process cannot name another process's memory, so it cannot
- **More memory than you have** - pages not in use can be evicted to disk and
- **Simplicity** - every program is compiled as if it owns all of memory; the

A **page fault** is the CPU trapping to the kernel because a page is not resident. A minor fault is resolved from memory (the page is already cached); a major fault requires disk I/O and is thousands of times slower. Sustained major faults are **thrashing** - the system spends all its time paging instead of working.

[Easy] Q. Stack vs heap?

Answer.

	Stack	Heap
Allocation	Move a pointer - very fast	Search a free list - slower
Lifetime	Automatic, at scope exit	Manual or garbage-collected
Size	Small and fixed (~1-8 MB)	Large
Fragmentation	None	Possible
Thread sharing	Private per thread	Shared

Local variables and call frames go on the stack; anything outliving its scope, or whose size is not known at compile time, goes on the heap. "Stack overflow" is deep recursion exhausting that small fixed region.

[Medium] Q. Explain the conditions for deadlock.

Answer. Four conditions must hold **simultaneously** (Coffman conditions); break any one and deadlock is impossible.

- 1 **Mutual exclusion** - a resource cannot be shared.
- 2 **Hold and wait** - a process holds one resource while waiting for another.
- 3 **No preemption** - resources cannot be forcibly taken back.
- 4 **Circular wait** - a cycle of processes each waiting on the next.

In practice you break **circular wait** by imposing a global lock ordering: every piece of code acquires locks in the same order, so a cycle cannot form. That is the answer to give - it is the one that is cheap and actually used.

Livelock is the related trap: threads are not blocked but keep reacting to each other and make no progress, like two people stepping aside in the same direction repeatedly.

[Medium] Q. What is the difference between concurrency and parallelism?

Answer. Concurrency is dealing with many things at once - a structure that lets tasks make progress in overlapping time periods. **Parallelism** is doing many things at once - literally simultaneous execution on multiple cores.

Concurrency is about program structure; parallelism is about hardware execution. A single-core machine can be concurrent (interleaving) but never parallel. An async web server is concurrent without necessarily being parallel.

The practical consequence: concurrency helps with **I/O-bound** work (waiting on network or disk), parallelism helps with **CPU-bound** work. Adding threads to a CPU-bound task on a single core makes it slower, not faster.

[Medium] Q. How does the kernel decide what runs next?

Answer. A scheduler balances throughput, latency and fairness. The approaches worth naming:

- **Round robin** - each ready thread gets a time slice in turn. Fair, simple,
- **Priority scheduling** - highest priority first; risks **starvation** of low
- **Multi-level feedback queue** - several queues; a task that uses its whole

Linux's CFS aims to give each runnable task an equal share of CPU time, tracking how much each has had and running whichever is furthest behind.

[Medium] Q. What is a system call?

Answer. The controlled entry point from user space into the kernel - the only way a program can do I/O, allocate memory or create processes.

Normal code runs in **user mode**, which cannot touch hardware or other processes' memory. A syscall traps into **kernel mode**, which can. The CPU enforces this boundary, which is what makes process isolation real rather than a convention.

Syscalls cost far more than a function call (a mode switch, argument validation, cache effects), which is why I/O is buffered - `printf` accumulates output and issues one `write` instead of one per character, and why `sendfile` exists to copy a file to a socket without crossing the boundary repeatedly.

[Medium] Q. What is a zombie process, why can it not be killed, and what is an orphan process?

Answer. A **zombie** is a process that has exited but still has an entry in the process table because its parent has not yet called `wait()` to read its exit status. It is already dead - it holds no memory, no CPU, no file descriptors - so there is nothing to kill; the entry exists only to hold the exit code until the parent collects it. An **orphan** is the mirror case: a process whose parent died first; it is alive and simply gets adopted by `init`/PID 1, which waits on it so it does not become a zombie when it exits.

Why zombies happen and what they mean:

- When a process exits, it must report **how** it exited (code or signal) to
- `kill` does not work on a zombie - there is no running code to receive the
- A single zombie is harmless (a few hundred bytes of table space). The

The practical rules you end with:

- Servers that fork workers should call `wait()` promptly, or set
- `init` (PID 1) exists partly to be the reaping backstop: every orphan is
- The classic confusion, stated out loud: `**zombie` = dead but uncollected